

Bases Linux

Commandes de bases



Stella Roulière

Objectifs



À l'issue de ce cours les apprenants seront capables de naviguer dans le système de fichiers Linux en utilisant des commandes essentielles telles que `pwd`, `cd`, et `ls`. Ils apprendront à gérer les fichiers et répertoires à travers des commandes comme `mkdir`, `cp`, `mv`, et `rm`. Ils comprendront également les concepts de chemins absolus et relatifs, ainsi que la gestion des métacaractères pour manipuler des fichiers efficacement.

De plus, ils acquerront des compétences sur l'identification des types de fichiers et l'utilisation de commandes pour modifier les attributs des fichiers.

Sommaire

3

Gestion des fichiers et dossiers

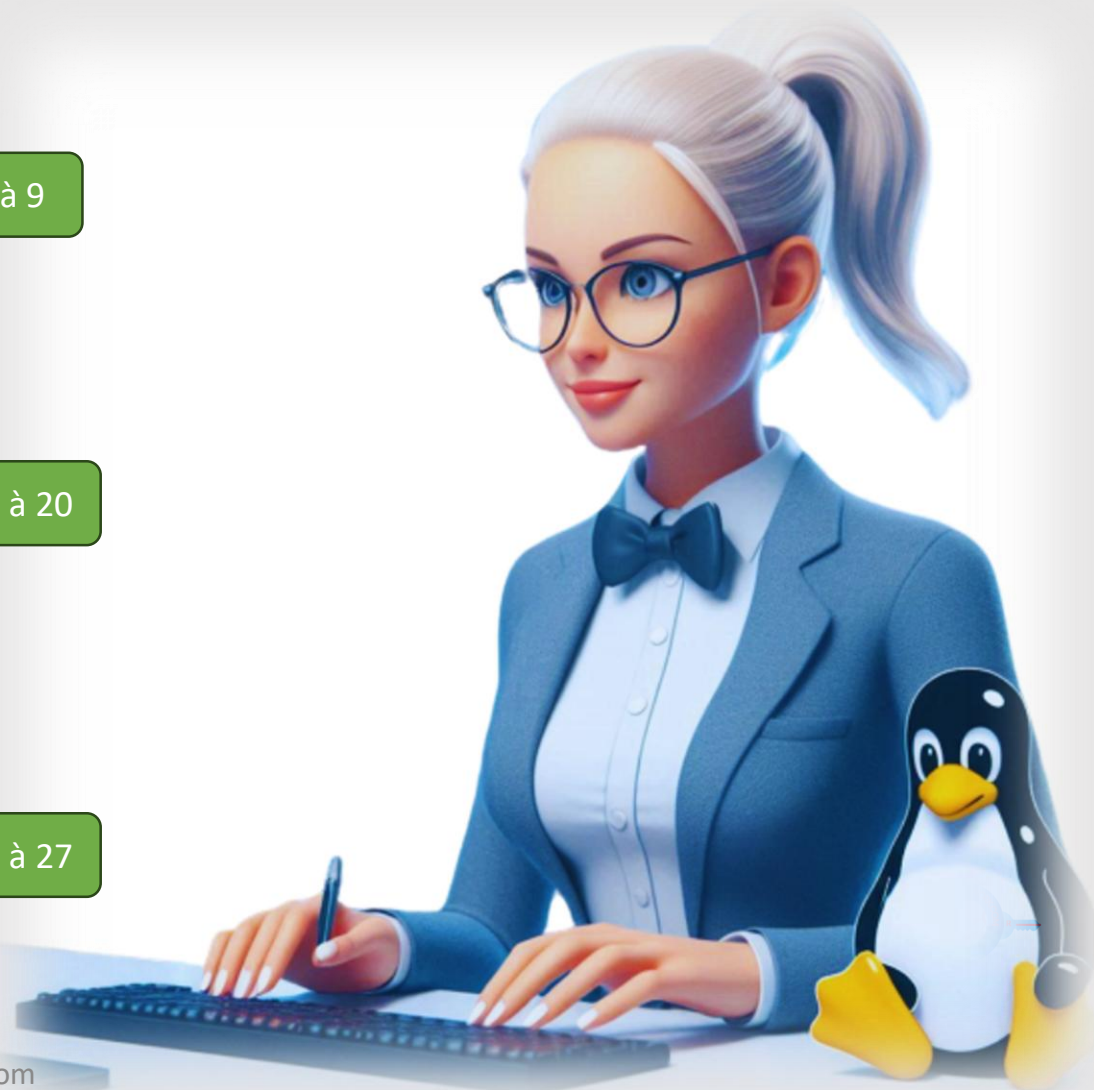
Page 4 à 9

**Les commandes: pwd, cd, mkdir, ls,
touch, cp, rm, mv**

Page 10 à 20

Les métacaractères

Page 21 à 27





Gestion des fichiers et dossiers



Chemin absolu et chemin relatif

Chemin absolu décrit l'emplacement exact d'un fichier ou dossier en partant toujours de la racine du système (qui est / sous Linux). Commence toujours par / (le slash de la racine).

Il indique le chemin complet, donc valide quel que soit le répertoire courant.

Pratique pour pointer vers un fichier dont l'emplacement est fixe et connu par tous les utilisateurs ou scripts du système.

```
stella@DEB-STELLA:~$ pwd
/home/stella
stella@DEB-STELLA:~$ cd /home/stella/Documents/
stella@DEB-STELLA:~/Documents$
```



Chemin absolu et chemin relatif

Un **chemin relatif** décrit l'emplacement d'un fichier ou dossier par rapport au répertoire courant (le dossier où on se trouve au moment de travailler).

Ne commence pas par / mais par un nom de répertoire, ./ pour le dossier courant, ou ../ pour le dossier parent.

Varie selon l'endroit d'où la commande est lancée, donc pas valide partout.
Souple pour des projets qui pourraient être déplacés ou copiés ailleurs, ou pour des scripts internes à un répertoire.

```
stella@DEB-STELLA:~$ pwd
/home/stella
stella@DEB-STELLA:~$ cd Documents/
stella@DEB-STELLA:~/Documents$
```



Chemin absolu et chemin relatif

Résumé visuel

	Chemin absolu	Chemin relatif
Point de départ	Toujours la racine /	Répertoire courant (.)
Apparence	Commence par /	Ne commence pas par /
Exemple	/home/stella/documents/courslinux.pdf	documents/courslinux.pdf
Portabilité	Identique partout dans le système	Dépend du dossier où l'on se trouve



Répertoire courant et répertoire parent

Sous Linux, l'organisation des fichiers suit une structure arborescente où chaque dossier peut contenir des sous-dossiers et des fichiers.

Répertoire courant

Le répertoire courant est le dossier dans lequel vous vous trouvez à un instant donné en ligne de commande. Noté par `.` (un simple point) dans les chemins relatifs et certaines commandes.

Quand vous lancez une commande sans indiquer de chemin, celle-ci s'applique par défaut au répertoire courant.

```
stella@DEB-STELLA:~/Images$ ls
image1.jpeg image2.jpeg image3.jpeg image4.jpeg
stella@DEB-STELLA:~/Images$ cp /tmp/fic.tmp .
stella@DEB-STELLA:~/Images$ ls
fic.tmp image1.jpeg image2.jpeg image3.jpeg image4.jpeg
stella@DEB-STELLA:~/Images$
```



Ici on indique de copier le fichier situé dans `/tmp` à l'endroit où on se trouve avec le point



Répertoire courant et répertoire parent

Répertoire parent

Le répertoire parent est le dossier qui se situe immédiatement au-dessus du répertoire courant dans l'arborescence. Noté par .. (deux points) dans les chemins relatifs et lors des déplacements avec la commande cd.

Il permet de remonter d'un niveau dans l'arborescence, accéder à des fichiers situés à un niveau supérieur, ou naviguer facilement.

```
stella@DEB-STELLA:~/Documents$ pwd
/home/stella/Documents
stella@DEB-STELLA:~/Documents$ cd ..
stella@DEB-STELLA:~$ pwd
/home/stella
stella@DEB-STELLA:~$
```



Types de fichiers

Quatre principaux types de fichier

Fichiers ordinaires :

Les fichiers ordinaires sont tous, des fichiers dans le sens usuel du terme, ainsi, les textes, les scripts, les commandes sont des fichiers ordinaires

```
stella@DEB-STELLA:~/Images$ ls -l
total 0
-rw-r--r-- 1 stella stella 0 30 nov. 13:15 fic.tmp
-rw-r--r-- 1 stella stella 0 30 nov. 12:41 image1.jpeg
```

Répertoires :

Le répertoire est pour le système un type de fichier particulier, contenant les références permettant d'accéder à des fichiers.

```
drwxr-xr-x 2 stella stella 4096 30 nov. 12:13 Backup
drwxr-xr-x 2 stella stella 4096 18 oct. 2023 Bureau
```

Fichiers spéciaux :

Les fichiers spéciaux désignent les périphériques matériels de la machine.

```
crw----- 1 root root 251, 0 30 nov. 10:37 rtc0
brw-rw---- 1 root disk 8, 0 30 nov. 10:37 sda
```

Liens symboliques :

Un lien symbolique est un fichier spécial qui permet de pointer vers un nom de fichier. Il permet de simplifier les accès aux fichiers et répertoires et à maintenir la compatibilité entre systèmes Unix.

```
lrwxrwxrwx 1 root root 3 30 nov. 10:37 cdrom -> sr0
```



Commande file

La commande file est un outil essentiel sous Linux pour identifier le type de fichiers.

Un fichier texte : `stella@DEB-STELLA:~$ file /etc/passwd`
`/etc/passwd: ASCII text`

Un répertoire : `stella@DEB-STELLA:~$ file /etc/`
`/etc/: directory`

Un lien symbolique: `stella@DEB-STELLA:~$ file /dev/cdrom`
`/dev/cdrom: symbolic link to sr0`

Un fichier spécial en mode bloc : `stella@DEB-STELLA:~$ file /dev/sda1`
`/dev/sda1: block special (8/1)`

Un script: `stella@DEB-STELLA:~$ file /etc/init.d/cron`
`/etc/init.d/cron: POSIX shell script, ASCII text executable`
`stella@DEB-STELLA:~$`



Nommage des fichiers

Caractères autorisés

En général, la plupart des caractères sont permis pour nommer vos fichiers. Le seul caractère strictement interdit est le slash (/) car il est réservé comme séparateur dans l'arborescence des dossiers.

La gestion de la casse sous Linux est un aspect important à prendre en compte lors de la manipulation de fichiers et de répertoires.

Contrairement à Windows, Linux est sensible à la casse par défaut, ce qui signifie que « **CoursLinux.pdf** », « **courslinux.pdf** » ou bien « **Courslinux.pdf** » sont considérés comme deux fichiers distincts

Les systèmes Unix, contrairement à Windows, ne s'appuient pas sur les extensions de fichiers pour déterminer leur type ou leur fonction.



Nommage des fichiers

Les fichiers dont le nom commence par un point (.) sont considérés comme cachés.

```
stella@DEB-STELLA:~$ ls -la
total 88
drwx----- 16 stella stella 4096 30 nov. 12:11 .
drwxr-xr-x  4 root  root  4096 18 oct. 2023 ..
drwxr-xr-x  2 stella stella 4096 30 nov. 12:13 Backup
-rw-----  1 stella stella  131 20 oct. 2023 .bash_history
-rw-r--r--  1 stella stella  220 18 oct. 2023 .bash_logout
-rw-r--r--  1 stella stella 3526 18 oct. 2023 .bashrc
drwxr-xr-x  2 stella stella 4096 18 oct. 2023 Bureau
drwx-----  9 stella stella 4096 20 oct. 2023 .cache
drwx----- 11 stella stella 4096 31 oct. 2023 .config
drwxr-xr-x  2 stella stella 4096 30 nov. 11:46 Documents
-rw-r--r--  1 stella stella 5290 18 oct. 2023 .face
lrwxrwxrwx  1 stella stella   5 18 oct. 2023 .face.icon -> .face
```

Les noms de fichiers peuvent contenir jusqu'à 255 caractères.



Les commandes:
pwd, cd, mkdir, ls,
touch, cp, rm, mv



Commande pwd

Lorsque vous exécutez `pwd` dans le terminal, il renvoie le chemin complet vers le répertoire actif.

Bien que la commande soit généralement simple, certaines implémentations peuvent offrir des options comme `-L` pour afficher le chemin logique ou `-P` pour afficher le chemin physique, en résolvant les liens symboliques

```
stella@DEB-STELLA:~$ pwd
/home/stella
stella@DEB-STELLA:~$ █
```



Commande cd

La commande `cd` (change directory) est un outil essentiel pour naviguer dans les systèmes de fichiers Linux et Unix.

```
stella@DEB-STELLA:/etc$ pwd
/etc
stella@DEB-STELLA:/etc$ cd /home/stella/
stella@DEB-STELLA:~$ pwd
/home/stella
stella@DEB-STELLA:~$ cd /var/cache/
stella@DEB-STELLA:/var/cache$ pwd
/var/cache
stella@DEB-STELLA:/var/cache$ █
```



Commande mkdir

La commande mkdir (make directory) permet de créer de nouveaux répertoires dans les systèmes Unix et Linux.

Options courantes

-p : Crée les répertoires parents si nécessaire

-m : Définit les permissions du nouveau répertoire (mkdir -m 777 Dossier)

```
stella@DEB-STELLA:~/Backup$ ls -l
total 0
-rw-r--r-- 1 stella stella 0 30 nov. 12:10 Backfile1.txt
stella@DEB-STELLA:~/Backup$ mkdir Année2024
stella@DEB-STELLA:~/Backup$ ls -l
total 4
drwxr-xr-x 2 stella stella 4096 30 nov. 13:28 Année2024
-rw-r--r-- 1 stella stella 0 30 nov. 12:10 Backfile1.txt
stella@DEB-STELLA:~/Backup$
```

```
stella@DEB-STELLA:~/Backup$ ls
Année2024  Backfile1.txt
stella@DEB-STELLA:~/Backup$ mkdir -p Année2025/janvier
stella@DEB-STELLA:~/Backup$ tree
.
├── Année2024
├── Année2025
│   └── janvier
└── Backfile1.txt

4 directories, 1 file
stella@DEB-STELLA:~/Backup$
```



Commande ls

La commande ls (list) est un outil essentiel dans les systèmes Unix et Linux pour afficher le contenu des répertoires.

Options courantes :

- l : Affichage détaillé (permissions, propriétaire, taille, date)
- a : Affiche tous les fichiers, y compris les fichiers cachés
- h : Tailles de fichiers lisibles (Ko, Mo)
- R : Affichage récursif des répertoires
- t : Trie par date de modification

Commande mkdir

```
stella@DEB-STELLA:~$ ls /etc/network/  
if-down.d if-post-down.d if-pre-up.d if-up.d interfaces  
stella@DEB-STELLA:~$
```

```
stella@DEB-STELLA:~$ ls -la  
total 684  
drwx----- 15 stella stella 4096 30 nov. 10:59 .  
drwxr-xr-x  4 root  root  4096 18 oct. 2023 ..  
-rw-----  1 stella stella  131 20 oct. 2023 .bash_history  
-rw-r--r--  1 stella stella  220 18 oct. 2023 .bash_logout  
-rw-r--r--  1 stella stella 3526 18 oct. 2023 .bashrc  
drwxr-xr-x  2 stella stella 4096 18 oct. 2023 Bureau  
drwx-----  9 stella stella 4096 20 oct. 2023 .cache  
drwx----- 11 stella stella 4096 31 oct. 2023 .config  
drwxr-xr-x  2 stella stella 4096 30 nov. 10:49 Documents
```



Commande ls -l

```
stella@Studi:~$ ls -l
total 4
drwxr-xr-x 3 stella stella 4096 11 févr. 16:16 toto
stella@Studi:~$
```

Diagram illustrating the output of the `ls -l` command, with components numbered 1 through 8:

Number	Component
1	Type de fichier (drwxr-xr-x)
2	Droits sur le dossier
3	Le 3 ième répertoire en partant de la racine (/home/stella/toto)
4	Nom de l'utilisateur propriétaire (stella)
5	Nom du groupe propriétaire (stella)
6	Taille en octets pour un fichier (4096)
7	Date de la dernière modification (11 févr. 16:16)
8	Le nom du répertoire en question (toto)

- 1-Type de fichier, dans mon exemple d comme dossier
- 2-Les droits sur le dossier « vu plus tard dans le cours
- 3-Le 3 ième répertoire en partant de la racine /home/stella/toto pour les fichiers le nombre de lien physique
- 4-Nom de l'utilisateur propriétaire
- 5-Nom du groupe propriétaire
- 6-Taille en octets pour un fichier
- 7-Date de la dernière modification
- 8-Le nom du répertoire en question



Commande touch

La commande touch est un outil polyvalent dans les systèmes Unix et Linux, principalement utilisé pour deux fonctions :

Modifier l'horodatage des fichiers :

Permet de mettre à jour la date et l'heure d'accès et de modification d'un fichier.

Par défaut, utilise l'heure actuelle du système.

Créer des fichiers vides :

Si le fichier spécifié n'existe pas, touch le crée automatiquement

Options principales

- a : Modifie uniquement le temps d'accès.
- m : Modifie uniquement l'heure de modification.
- c : Empêche la création d'un nouveau fichier.
- d ou --date=STRING : Utilise un horodatage spécifié.
- t : Permet de définir un horodatage manuel au format [AA]MMJJhhmm[.ss].
- r : Utilise l'horodatage d'un fichier de référence

```
stella@DEB-STELLA:~/Backup/Année2024$ touch compta2024.txt
stella@DEB-STELLA:~/Backup/Année2024$ ls -l
total 0
-rw-r--r-- 1 stella stella 0 30 nov. 13:38 compta2024.txt
stella@DEB-STELLA:~/Backup/Année2024$
```



Commande cp

La commande cp (copy) est un outil essentiel dans les systèmes Unix et Linux pour copier des fichiers et des répertoires.

cp [options] source destination

```
stella@DEB-STELLA:~/script$ ls
bouclefor.sh  case2.sh  color.sh          fonction.sh.save  image2  random
calculIP.sh   case.sh   ficexbouclefor.txt image             model.sh  test.
stella@DEB-STELLA:~/script$ cp color.sh color.sh.bk
stella@DEB-STELLA:~/script$ ls
bouclefor.sh  case2.sh  color.sh          ficexbouclefor.txt  image  model.sh
calculIP.sh   case.sh   color.sh.bk       fonction.sh.save     image2  randomwhile.
stella@DEB-STELLA:~/script$
```

```
stella@DEB-STELLA:~/script$ cp * ../Doc-perso/
```

Options courantes

- r ou -R, copie récursivement des répertoires et leur contenu
- p conserve les permissions, propriétés et timestamps des fichiers
- i : Demande confirmation avant d'écraser un fichier existant.
- u : Copie uniquement si la source est plus récente que la destination.



Commande rm

La commande **rm** (remove) est utilisée dans les systèmes Unix et Linux pour supprimer des fichiers et des répertoires. Utiliser **rmdir** pour les dossier vide.

rm [options] fichier(s)

```
stella@DEB-STELLA:~/script$ ls ../Doc-perso/
bouclefor.sh  case2.sh  color.sh      ficexbouclefor.txt  image  model.sh      test.sh  TP2.sh
calculIP.sh   case.sh   color.sh.bk   fonction.sh.save    image2  ramdomwhile.sh TP2-B.sh  whileread.sh
stella@DEB-STELLA:~/script$ rm ../Doc-perso/
rm: impossible de supprimer '../Doc-perso/': est un dossier
stella@DEB-STELLA:~/script$
```

```
stella@DEB-STELLA:~/script$ rm -r ../Doc-perso/
stella@DEB-STELLA:~/script$ ls ../Doc-perso/
ls: impossible d'accéder à '../Doc-perso/': Aucun fichier ou dossier de ce type
stella@DEB-STELLA:~/script$
```

Options courantes

- r** ou **-R** : Suppression récursive (pour les répertoires)
- f** : Force la suppression sans demander de confirmation
- i** : Mode interactif, demande confirmation avant chaque suppression
- v** : Mode verbeux, affiche les actions effectuées



Commande mv

La commande mv (move) est un outil essentiel dans les systèmes Unix et Linux pour déplacer et renommer des fichiers et des répertoires.

mv [options] source destination

Options courantes

- i** : Mode interactif, demande confirmation avant d'écraser
- f** : Force le déplacement sans demander de confirmation
- u** : Déplace uniquement si la source est plus récente que la destination
- v** : Mode verbeux, affiche les actions effectuées



Commande mv (déplacer)

```
stella@DEB-STELLA:~$ ls
Backup  Documents  fic2.txt  Modèles  Public  Vidéos
Bureau  fic1.txt   Images   Musique  Téléchargements
stella@DEB-STELLA:~$ ls Backup/
stella@DEB-STELLA:~$ █
```

```
stella@DEB-STELLA:~$ mv fic1.txt Backup/
stella@DEB-STELLA:~$ ls
Backup  Documents  Images  Musique  Téléchargements
Bureau  fic2.txt   Modèles  Public  Vidéos
stella@DEB-STELLA:~$ ls Backup/
fic1.txt
stella@DEB-STELLA:~$
```



Commande mv (renommer)

```
stella@DEB-STELLA:~/Backup$ ls  
fic1.txt  
stella@DEB-STELLA:~/Backup$ mv fic1.txt Backfile1.txt  
stella@DEB-STELLA:~/Backup$ ls  
Backfile1.txt  
stella@DEB-STELLA:~/Backup$ █
```



Les métacaractères



Les métacaractères

Les métacaractères sont des symboles spéciaux utilisés dans les expressions régulières (regex) pour définir des motifs de recherche complexes.

Métacaractères de base

. : Correspond à n'importe quel caractère sauf le saut de ligne.

^ : Indique le début de la chaîne ou de la ligne.

\$: Marque la fin de la chaîne ou de la ligne.

*** : Répète 0 ou plusieurs fois l'expression précédente.**

+ : Répète 1 ou plusieurs fois l'expression précédente.

? : Répète 0 ou 1 fois l'expression précédente.

| : Opérateur "ou" pour les alternatives



Les métacaractères

Délimiteurs et groupes

[] : Définit une classe de caractères.

[...] 1 seul caractère parmi ceux de la liste entre []

[...-...] 1 seul caractère compris dans l'intervalle de caractères, bornes incluses

[^...] négation, 1 seul caractère autre que ceux spécifiés entre [] (! est aussi valable [!...])

() : Crée un groupe de capture.

{ } : Spécifie un nombre précis de répétitions.

Caractères d'échappement

**** : Échappe un métacaractère pour lui donner sa valeur littérale.

Métacaractères dans les classes de caractères

^ : Négation de la classe (uniquement si placé au début).

- : Indique un intervalle de caractères.



Les métacaractères

Exemples

```
stella@DEB-STELLA:~$ cd /etc/
stella@DEB-STELLA:/etc$ ls *.conf
adduser.conf          fuse.conf             mke2fs.conf          sudo.conf
apg.conf              gai.conf              nftables.conf        sudo_logsrvd.conf
appstream.conf        host.conf             nsswitch.conf         sysctl.conf
ca-certificates.conf  kernel-img.conf      pam.conf              ucf.conf
debconf.conf          ld.so.conf            reportbug.conf        usb_modeswitch.conf
deluser.conf          libao.conf            resolv.conf           xattr.conf
discover-modprobe.conf libaudit.conf         rygel.conf
e2scrub.conf          logrotate.conf        sensors3.conf
stella@DEB-STELLA:/etc$ ls xa?t?.conf
xattr.conf
stella@DEB-STELLA:/etc$
```



Les métacaractères

Exemples [A-Za-z] 1 caractère parmi toutes les lettres minuscules et majuscules

ab[!aei]h tous les noms de quatre caractères qui commencent par "ab" et se terminent par "h" dont le troisième caractère n'est pas "a" ou "e" ou "i"

```
stella@DEB-STELLA:/etc$ ls *.conf
adduser.conf          fuse.conf             mke2fs.conf          sudo.conf
apg.conf              gai.conf              nftables.conf        sudo_logsrvd.conf
appstream.conf        host.conf             nsswitch.conf         sysctl.conf
ca-certificates.conf  kernel-img.conf      pam.conf              ucf.conf
debconf.conf          ld.so.conf            reportbug.conf        usb_modeswitch.conf
deluser.conf          libao.conf            resolv.conf           xattr.conf
discover-modprobe.conf libaudit.conf         rygel.conf
e2scrub.conf          logrotate.conf        sensors3.conf
stella@DEB-STELLA:/etc$ ls de[b][c][s][!a][!e][!i]*.conf
debconf.conf  deluser.conf
stella@DEB-STELLA:/etc$
```



Les métacaractères

Exemples

Comme pour les caractères, il est possible d'offrir un choix d'expressions, en utilisant le caractère accolade :
`{chaine1,chaine2,chaine3}` chaine1 ou chaine2 ou chaine3

```
stella@DEB-STELLA:~/Images$ ls
stella@DEB-STELLA:~/Images$ touch image{1,2,3,4}.jpeg
stella@DEB-STELLA:~/Images$ ls
image1.jpeg  image2.jpeg  image3.jpeg  image4.jpeg
stella@DEB-STELLA:~/Images$ █
```



Les métacaractères

Exemples

En plus des métacaractères, le Shell interprète de façon particulière certains caractères appelés caractères spéciaux.

\$ permet d'afficher le contenu d'une variable avec la commande echo

Simple quote ‘ ‘

```
stella@DEB-STELLA:~$ echo '$HOME'
$HOME
```

Double quote “ ”

```
stella@DEB-STELLA:~$ echo "$HOME"
/home/stella
stella@DEB-STELLA:~$ █
```



Les métacaractères

Exemples

Quelques caractères d'échappement

\' : Apostrophe

\" : Guillemet double

**** : Barre oblique inversée

\n : Nouvelle ligne

\r : Retour chariot

\t : Tabulation horizontale

\b : Retour arrière (backspace)

\f : Saut de page (form feed)

\v : Tabulation verticale

\a : Alerte sonore (bell)

\? : Point d'interrogation

```
stella@DEB-STELLA:~$ echo -e "Je suis située dans $HOME et le nom de ma machine est $HOSTNAME, alors maintenant utilisons les caractères d'échappement, \n je viens de revenir \t à la ligne avec une tabulation."
Je suis située dans /home/stella et le nom de ma machine est DEB-STELLA, alors maintenant utilisons les caractères d'échappement,
je viens de revenir      à la ligne avec une tabulation.
stella@DEB-STELLA:~$
```




Mise en situation

16/03/2026



cours.it-system-network.Com